

Research Assistant (HiWi) Challenge: RGB-D Scene Graph Generation with BIM/IFC Priors

Duration: 2 Weeks

AI Policy: Open AI Usage. *You are fully permitted and encouraged to use AI tools (e.g., LLMs, GitHub Copilot) for code generation, debugging, and conceptual brainstorming. However, you must deeply understand, properly document, and be prepared to scientifically defend your pipeline and architectural choices.*

1. Background & Objective

Constructing actionable spatial representations is a core challenge in industrial robotics and spatial computing. While visual-only scene graph generation from RGB-D sequences is an active area of research, industrial environments often provide rich Digital Twins or Building Information Models (BIM) in the form of Industry Foundation Classes (IFC) data.

Your Objective: Design and implement a pipeline that generates a semantic 3D scene graph from an egocentric RGB-D sequence. *Crucially, you should incorporate structural and topological priors extracted from the provided IFC file to enrich or constrain the visual scene graph.*

Important Disclaimer: This is an open-ended research problem. **The goal is not to deliver a flawless, fully solved pipeline.** Rather, we want to evaluate your scientific thinking, your approach to integrating multimodal spatial data, and your code engineering habits. **Proper documentation, theoretical justification, and critical analysis of failure modes carry a significantly higher weight than a perfectly functioning system.** A partial solution with exceptional architectural documentation is strongly preferred over a complete but poorly explained codebase.

2. Provided Inputs

1. **Visual Observation:** A short, egocentric RGB-D video sequence capturing an indoor environment.
2. **Camera Trajectory:** A sequence of corresponding camera poses for each frame, already aligned to the global IFC coordinate system (simplifying the SLAM/registration overhead).
3. **Architectural Prior:** The corresponding .ifc file and pointcloud of the environment. *(Hint: You can generate downstream data such as floor plan if you convert pointcloud to a BEV Map)*

3. Core Tasks & Methodology

You are encouraged to approach this challenge pragmatically, using off-the-shelf vision models rather than training from scratch.

Task A: Visual Scene Graph Generation (The Primary Task)

- **Objective:** Process the RGB-D sequence and known camera poses to instantiate a 3D scene graph $G = (V_{obj}, E_{rel})$.
- **Methodology:** Utilize existing state-of-the-art frameworks (e.g., DAAAM, PIX2Graph, or a custom combination of 2D object detection, zero-shot segmentation, and depth projection).
- **Output Requirements:** Nodes should represent localized objects (e.g., 3D centroids or bounding boxes). Edges should represent spatial heuristics (e.g., proximity, nearest-neighbor).

Task B: Incorporating BIM Priors (The Caveat)

- **Objective:** Enrich the dynamically generated visual scene graph by leveraging the static architectural prior.
- **Methodology:** Use IfcOpenShell to programmatically extract spatial boundaries (e.g., IfcSpace) and connective portals (e.g., IfcDoor) from the .ifc file.
- **Integration:** Propose and implement a method to fuse this structural prior with the visual graph. For example, map the localized 3D object centroids into their corresponding IFC rooms, creating hierarchical bipartite edges (Room $\rightarrow$ Object) using simple geometric inclusion logic.

4. Deliverables & Evaluation Criteria

Please submit a source code repository containing reproducible scripts, environment requirements (e.g., requirements.txt or Dockerfile), and a comprehensive technical README.md.

Evaluation Criteria:

1. **Scientific Reasoning & Documentation (50% - Highest Weight):** The quality of your README.md. You must formally justify your chosen methodologies, explicitly document why certain approaches failed, acknowledge the limitations of your implementation, and outline how the pipeline *should* work ideally.
2. **Integration & API Proficiency (30%):** Your ability to set up and run off-the-shelf vision models, as well as your effective use of IfcOpenShell to extract geometric data.
3. **Data Structures & Spatial Logic (10%):** The efficiency of your 3D spatial indexing, coordinate transformations, and graph representation (e.g., NetworkX).
4. **Code Completeness (10%):** The degree to which the pipeline successfully runs end-to-end.